

Enhancing Phishing Website Detection with Machine Learning

AP Research

Word Count: 4507

Enhancing Phishing Website Detection with Machine Learning

With over 5.5 billion users (more than two thirds of the global population), the internet is one of the biggest spaces where people share ideas, communicate, and collaborate. Daily, these people access websites for various purposes, including interacting with others, buying or selling products, entertainment, or even just gaining information from blogposts and news articles. However, the multitude of facilities on the internet is threatened by cybercriminals, who out of multiple types of cyberthreats, also build malicious websites, websites that are intentionally designed to deceive users, steal sensitive information, or distribute harmful software. While malicious websites can have multiple intents, the most common ones include obtaining personal biographical information of the victim, obtaining their payment details such as credit/debit card information and social security number, or downloading malware onto their computer.

As online activity continues to grow, so does the opportunity for malicious actors to exploit users through more sophisticated, deceptive tactics. Among the various attacks, social engineering-based attacks are proven effective because they target humans rather than more technical systems. These attacks rely on carefully crafted online content that causes ordinary users to unintentionally give up private information, making it especially difficult for users without assistance, such as young children or older adults. Consequently, researchers and cybersecurity professionals have placed emphasis on understanding how such threats emerge and persist, and how they can be detected before impact. One of the most damaging forms of deception that is analyzed is phishing, where malicious actors pretend to be from reputable companies to build trust in their target to reveal their personal information. This focus is well-justified, as the 2024 Verizon Data Breach Investigation Report (DBIR) reports over 68% of breaches involving a human element, such as

clicking on a malicious link, or mistakenly sharing credentials to a treat actor, demonstrating the prevalence of this type of attack among the other cybersecurity threats.

Literature Review

Phishing Websites as a Scamming tool

Phishing and spoofing remain pervasive yet evolving threat in the field of cybersecurity and data safety, causing significant financial harm. According to the FBI, spoofing is a tactic used by cybercriminals to disguise their email address, sender name, or phone number, by often changing a few letters or symbols, to disguise themselves as a trusted source (FBI). Additionally, according to the Federal Trade Commission, scammers often tell a story to trick their targets into clicking on a link or an attachment; some messages being “they’ve noticed some suspicious activity and log-in attempts” or that “there is a problem with your account or payment information” (FTC, 2025). Messages like these often prove to be effective for scammers to gain their targets’ trust. In fact, over 73% of Americans surveyed by the Pew Research Center reported experiencing an online fraud or attack in the 2024 (Anderson et al., 2025). Meanwhile, the FTC reported over \$12.5 billion in fraud in the same year, of which \$3 billion being lost only to impostor frauds (which include spoofing and phishing-like attacks) (FTC, 2025). With the immense losses people experienced due to fraud, it is important to address the issue of phishing websites.

Traditional approaches to targeting Phishing Websites

Traditionally, many phishing website detection software typically found in browsers such as Microsoft Edge, Firefox, and Google Chrome utilize List Based methods to detect Phishing Websites. Safi et al, researchers from Nangarhar University, describe some of these methods: List based methods work by filtering out a dataset of known fraudulent websites to ensure users’ safety.

However, slight modifications on the website URLs – Uniform Resource Locators, or the link used to access any webpage on the internet – made by scammers can bypass such List based techniques (2023). Another traditional approach is to analyze the visuals of the websites and compare them to previously visited web pages, and the pages the spoofed websites are trying to replicate. However, they are unable to detect zero-hour phishing attacks, attacks that exploit new and previously unknown vulnerabilities. (2023)

Increasingly, cybersecurity specialists have begun to build and train machine learning models for their ability to learn patterns and detect fraud websites flexibly, even though attackers employ newer tactics. Machine learning models, also referred to as ML models, are computational systems that learn patterns from data to make predictions or classifications without being explicitly programmed. Bitaab et al., researchers at Arizona State University, Amazon, and X Corp, proposed Scam Magnifier, a system that utilized the powers of machine learning to track fraud websites in real-time with high accuracy. While many of such ML models have been created in the past, they differ in their accuracy and false-positive rates. By optimizing such accuracy and minimizing false positives, these machine learning models can provide a much more powerful solution against phishing websites compared to traditional approaches such as list-based and visual-similarity approaches.

Features analyzed in ML Phishing Website Detection

In the field of cybersecurity research, phishing websites can be a vast field to explore, comprising of numerous types of data researchers can analyze to make the most optimized and accurate machine learning classifier. Alkawaz et al., researchers and faculty from the Management and Science University, Malaysia, conducted a literature review to identify current approaches to phishing detection. They identified and categorized phishing detection, specifically software

detection, into 4 main categories: Blacklists, Heuristics, Visual Similarity, and Machine Learning (2021). They then show how the work of other researchers falls under one of these four primary categories and conclude how machine learning is the most effective out of the four at effectively classifying phishing and legitimate websites, emphasizing the scope of research that is still to be done (2021).

Challenges and Gaps

While many researchers have been successful in creating highly accurate machine learning models, many of them used limited data sets, and only analyze 2-3 variables when building their model. Adane et al., researchers from Abra Minch University, conducted a systematic review of 30 papers to identify critical gaps in phishing detection research. Their analysis highlighted several recurring limitations in papers, specifically regarding dataset selection and size, ratio of phishing and legitimate website data, the dataset train-test split ratio, and the type of website features selected and analyzed (2022). While many issues were identified among these sources, this research paper focuses on issues with the dataset size adequacy (how large of data ML models should be trained on to produce the best results). Datasets are a foundational factor which can determine how a ML model functions, as their underlying pattern is what the models try to analyze. However, the current provides inconsistent guidance on how datasets should be effectively scaled, specifically to make the most optimum predictions for classifying phishing websites. By addressing this gap, this research paper aims to answer the question, **how can the accuracy of machine learning classifiers be improved and leveraged for detection of phishing websites?**

The Problem surrounding Dataset Scaling

While many researchers use a moderately sized dataset for the purposes of training their ML models, the use of too-small or too-large datasets might disrupt their findings and alter the results to be “too good” or “too bad.” In a research study conducted by Devi et al. on the various types of machine learning algorithms, the researchers used a dataset with 10,000 URLs to train their models and understand the performance of each (2025). However, training models with a small number of datapoints, in this case, only 10,000, may lead to overfitting: a concept in machine learning that occurs when ML models learn the entire dataset instead of observing and learning the patterns within each, rendering it to produce inaccurate results. For this research paper, the lack of many datapoints may have affected the accuracy of the logistic regression model, which was only 94% (when compared to more commonly implemented detection systems with accuracies between 97-99%).

On the other hand, datasets that have too many data points, when taught to ML models, can lead to underfitting: a concept in machine learning where a model is unable to identify the underlying pattern of the training dataset and get confused among all the data that acts as ‘noise’, leading it to make assumptions based on a very common pattern. In a research study by Owa et al. on testing multiple machine learning algorithms with large test data, the use of over 640,000 URLs only leads to an accuracy between 86% and 87%, which is not a good score for phishing website detection software (2025). Given that both too small and too large datasets can lead to results that may be misleading or not optimum, it is necessary to find the approximate range for dataset sizes that optimize performance scores, while preventing underfitting and overfitting of the model.

Methods

To help determine the most optimal size of machine learning models for phishing URL detection that avoid both underfitting and overfitting, I took an experimental and creative

approach. Performing an experimental approach allows me to manipulate the variables (training dataset size) to identify a trend in dataset scaling versus model performance. Additionally, undertaking a creative approach required me to train multiple machine learning models, analyzing the performance of which allowed me to witness the impact of increasing and decreasing the dataset size on the model's accuracy and to understand the range of sizes that yield the best results.

Dataset Identification and Preparation

One of the preliminary steps in training a machine learning model is to identify the dataset that will be used. In their research paper, Bitaab et al. used a crowdsourcing technique by using Reddit to crawl through various posts to identify suspicious website URLs over the span of 3 year to acquire roughly 9000 live URLs. Since curating a comprehensive dataset of phishing website URLs and their associated features can be labor-intensive, time-consuming, and potentially costly, this study utilizes a preexisting open-source phishing dataset. While numerous datasets can be found on the internet that are compiled by trustworthy researchers, they differ in their sizes and the number of features they have information about. Two of the most comprehensive and reputable datasets I identified were PhiUSIIL and Mendeley, both of which contain large amounts of labeled phishing and legitimate website URLs and have been used by various researchers, such as Sarasjati et al. 2022 and Vajrobol et al., showing its usage in the field of phishing research. The expansive size of these datasets was most optimum for this research, as it would enable me to test the performance of models trained on larger dataset sizes.

However, these datasets differ in their size and the number of features they offer. The PhiUSIIL dataset contains over 235,000 unique URLs, and information about 54 characteristics of each URL, such as the URL length, number of obfuscated characters, number of digits, and has the keyword “pay”, etc. On the other hand, the Mendeley dataset has over 450,000 unique URLs,

but only has two columns of data: the URL itself and its label (phishing or legitimate). Despite having no features, I chose to use the Mendeley dataset solely because of its size, as it would allow me to train ML models of sizes up to 450k rows, as opposed to just 235k rows.

Though I have many rows of data, the ML models can only be trained and tested on numerical form of data, which website URLs are not. To solve this problem, I decided to use BERT (Bidirectional Encoder Representations from Transformers), an open-sourced framework developed by researchers at Google that can perform tasks relating to natural language processing – a branch of artificial intelligence that enables computer understanding human language. BERT works by converting any human text into numerical and symbolic “chunks” known as tokens, while encapsulating the context of the surrounding text with each token. For my research, Bert was especially useful as I was able to convert each URL into a long vector of numerical values that cannot be understood by a human but can be understood by computers. Using Bert alongside the Mendeley dataset allowed me to leverage the vast amount of data to get data that is machine understandable. Appendix A shows the Python (programming language) program I used to obtain the full dataset with a vectorized form of each URL.

Training Machine Learning Models

After the dataset is prepared, the next step is to train and test the models. Though this process can be complex and time consuming, Scikit-learn is a package built within the Python programming language that provides programmers with the tools required to easily perform machine learning-related tasks. For my research, I decided to use the python programming language, especially because of the tools such as Scikit-learn is provides, that would enable me to perform large scale machine learning tasks in less amount of time. The steps that were required to

write the program (an example of which can be found in Appendix B) using Scikit-learn are outlined below:

Load and Randomly Select a portion of the data (Appendix B.i) – The previously obtained dataset contains all the data that I would need for the research. However, to train a model of different sizes, I needed to randomly select a portion of the dataset of size ‘n’. To ensure I trained models with a variety of sizes of datasets, I chose the following values of ‘n’ to train the models upon: 1k, 5k, 10k, 25k, 50k, 100k, 200k, 300k, and the full data. Additionally, to eliminate any biases in picking the data, I used a function to randomly choose ‘n’ rows of data from the full dataset, confirming that the new dataset has a mix of both phishing and legitimate rows of data.

Train-test Split the Data (Appendix B.ii) – For the new dataset of size ‘n’ that was created, not all the data will be used to train the model; rather, some of it will be used to test the model. For all models, I decided to use an 80-20 train-test split ratio, which means that 80% of the dataset is used to train the model, and the rest 20% will be used to test the model later. I used this specific ratio as it is the most common one, and it generally produces more effective outputs than other ratios. One of the methods provided by Scikit-learn is the train-test split method, which divides the data into 4 categories: Train-x, Train-y, Test-x, and Test y. In these four categories, train represents the data used in the training process, test represents the data used in the testing process, x is the independent variable (the tokenized vector of each URL), and y is the dependent variable (the label of the URL – 0 or 1, legitimate or phishing).

Train the model (Appendix B.iii) – In machine learning, there are various types of algorithms that use different types of logic to make predictions, which can also lead to each having different accuracies. Based on prior experimentation, Logistic regression is an algorithm that performed slightly better than other common algorithms. To ensure the algorithm used does not

skew the results of the research, I used a Logistic Regression algorithm for all trials. Scikit-learn also provides access to this algorithm, using which I can input the Train-x and Train-y data to obtain a model that is temporarily stored in the instance of the program running. This same model can be used later to make predictions.

Test the model (Appendix B.iv) – After the model is trained, the next step is to input the test-x data and see the model's predictions. These predictions can then be compared against the actual labels (test-y) to see how accurate the model's predictions were. Scikit-learn provides tools to view different metrics about the model's performance: Precision, Recall, and F1. Though the different metrics tend to have similar scores, they reveal different information in the accuracy of the model. Precision measures how many of the websites the model flagged as phishing were actually phishing, which is important to reduce false alarms. Recall measures how many actual phishing websites the model successfully identified, which is critical for minimizing missed threats. The F1 score combines precision and recall into a single metric, providing a balanced view of model performance, especially when the classes are imbalanced. (Tigerschiold, 2022) These three metrics were compared between those of the other models to analyze if there was a significant difference in the metrics of each.

To reduce experimental uncertainty to some extent, I ran the program 5 times for each dataset size with a different random state for each. The random state is a parameter in machine learning that acts like the seed for the pseudo-random number generator being used to perform functions related to randomness. Using the same random state repeatedly would produce the exact same results as all trials, allowing a machine learning model to be reproducible, so anyone can get the exact results I did with the random state I used. However, to ensure that a specific random state

is not skewing my data, running the program with different random states would ensure that the performance of a model at a specific dataset size is not too good or too bad purely by chance.

Results & Discussion

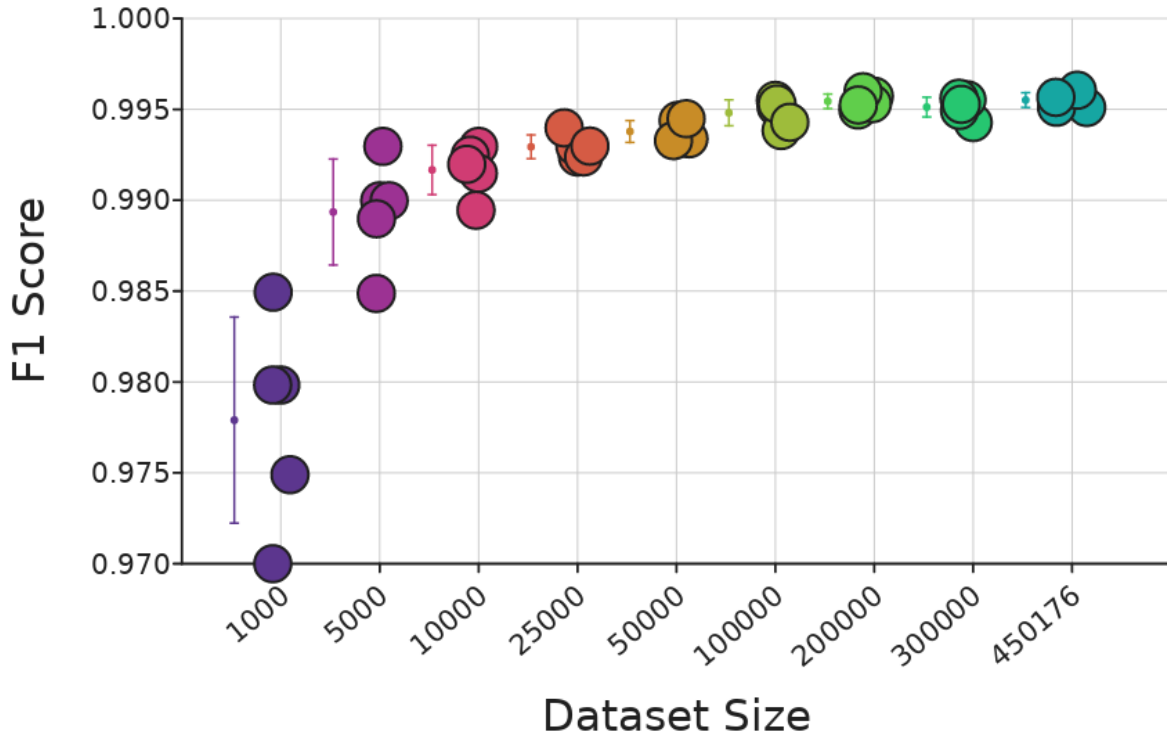
For performing statistical analyses, the entire raw data was used, which can be found in Appendix C. Since I trained and tested 5 separate machine learning models with different random states, Table 1 shows the mean and standard deviation of the F1 score for each dataset size. Although I collected data for all metrics, I specifically looked at the F1 scores because F1 scores combine precision and recall into a single value using a harmonic mean, which is especially useful when the dataset has imbalanced classes (in this case, the difference in the phishing and legitimate URLs in the dataset). Additionally, the values for each metric were extremely close to each other, so performing statistical analysis on only one metric would be more feasible and useful, instead of juggling with multiple variables at once.

Table 1: Mean and Standard Deviation of F1 Scores for different Dataset Sizes		
Dataset Size	Mean	Standard Deviation
1000	0.9779	0.0056662
5000	0.989349	0.00291642
10000	0.991667	0.00135894
25000	0.992939	0.000654643
50000	0.993782	0.000600344
100000	0.99481	0.000712179
200000	0.995441	0.000399021
300000	0.995125	0.00054198
450176	0.99551	0.000402291

Based on Table 1, a positive trend can be seen between the dataset size and the mean value of each metric, showing how there is a generally positive relationship between dataset size and the performance of a ML model. However, it can also be noted that the standard deviation values are

higher for smaller dataset sizes as compared to large dataset sizes, showing how the performance becomes slightly less uniform when machine learning models are trained on smaller datasets.

Figure 1: F1 scores of ML Models across different Dataset Sizes



However, a clearer relationship can be seen when these data points are plotted. The magnitude of the spread of the datapoints and their relative means can be observed much better in the graph in *Figure 1*, where a more apparent logarithmic relationship can be identified. A logarithmic regression in the form of $y = a + b \ln(x + c)$ was created, where x represents the training dataset size and y represents the average F1-score of the 5 classifiers trained on the dataset. The resulting relationship can be represented by the function:

$$y = 0.966366 + 0.00241469 \ln(x)$$

This regression had a R^2 statistic of 0.7753, meaning that this regression does not completely align with the data, but is able to capture $\sim 77\%$ of its patterns. Though this regression does not explain all the variation in data, it still represents a strong correlation, suggesting that dataset size is a significant factor influencing model performance, even if other variables also contribute. However, this also reveals that there might be other factors apart from just dataset size that affect an ML model's performance.

Though the mean and standard deviation only provide a general understanding of this trend, it is important to conduct statistical analyses to test whether the difference in F1 scores for the dataset sizes is significant. The specific statistical test I chose was an ANOVA test, as my independent variable, dataset size, was categorical given the discrete values I chose, and my dependent variable, F1 score, was a continuous decimal number between 0 and 1. Performing an ANOVA test combined with a post-hoc test would allow me to identify if the difference in accuracy values between each dataset size was statistically significant, and if they were, which size was the one that was different. After performing an ANOVA test, I received a p-value of less than 0.01, which means that with 99% confidence, the likelihood of the categories being different purely by chance is less than 1%. This indicates that there is some statistically significant difference between the accuracy metrics of each dataset size.

To understand which dataset sizes produce significantly different results from each other, a post-hoc test was conducted to compare every category with every other category to determine if there is a statistically significant difference between them. Appendix D shows the specific p-values of the categories compared against each other, for the F1 metric.

Based on these p-values and a confidence level of 95%, it can be assumed that if there is a p-value less than 0.05, there is a significant difference between the two groups. From Appendix D,

a p-value less than 0.05 can be observed for the comparisons of the performance of ML models with a dataset size of 1000 to all other sizes. In the current context, this means that the accuracy of machine learning models trained and tested on 1000 URLs is significantly less than that of models using a larger dataset. A p-value less than 0.05 can also be seen when comparing the performance of ML models with dataset size of 5000 to those with a dataset size of 100,000 or greater. In this context, it means that the accuracy of machine learning models trained and tested on 5000 URLs could be similar those with a dataset size of 10000, 25000, and 50000 URLs, but a significant difference can be seen when compared with more than models with a dataset size of 100,000 or more.

Conclusion

Based on the results of this study, a clear and consistent relationship between dataset size and machine learning model performance can be observed. As the dataset size increased, the F1 score improved while the variability in performance, as measured by standard deviation, decreased. This indicates that larger datasets lead to better-performing models, with outcomes that are more stable and reliable. In particular, very small datasets such as the one with 1000 URLs resulted in significantly worse performance when compared to the other larger datasets, highlighting the limitation of insufficient training data in crucial classification tasks such as phishing detection.

These conclusions are supported by statistical analysis. The ANOVA test confirmed that the difference in model performance across dataset sizes is statistically significant, and the post-hoc test further indicated the specific dataset sizes for which the differences were notable. While increasing the dataset size from 1000 to 5000 and beyond yields substantial improvements in performance, the marginal gains begin to diminish as the datasets become larger. For instance, the

difference between the performance of models trained on 100,000, 200,000 and larger dataset size are not statistically significant, suggesting that beyond a threshold, increasing data size provides limited additional benefit. This introduces the idea of a “saturation point”, where the cost of acquiring and processing more data can outweigh the potential improvements gained in model accuracy.

Limitations

Despite the valuable insights on dataset size, there are several limitations that must be acknowledged. First, this study was conducted using only a single dataset of phishing and legitimate URLs, the Mendeley dataset. As a result, these findings may vary slightly or even significantly when implemented on other datasets that differ in structure and feature representation. Different datasets may contain varying levels of noise, features about a URL, or the type of data they have, all of which can influence model performance. Thus, it may also be possible that dataset size affects performance of machine learning models with only this dataset.

Second, the models in this study were trained on BERT-vectorized representations of URLs rather than raw or manually engineered features. While this approach leverages powerful natural language processing techniques, its feasibility in real-time phishing detection could raise questions, especially due to the significant time taken to vectorize each URL. Additionally, this method ignores certain interpretable features about a URL, instead focusing on the numeric embedding obtained for BERT, which may also play a role in classification accuracy. However, BERT embeddings were necessary here as logistic regression models only work with data in numerical format, instead of regular text.

Another limitation lies in the diversity of models used. Although multiple models were trained with different random states as different trials to ensure random splits between training and testing datasets, the study did not explore a wide range of model architectures or hyperparameter configurations. For instance, all models used a logistic regression algorithm to train and test the datasets over, while other algorithms such as support vector machines, decision trees, and neural network were not analyzed for consistency. Using a different algorithm could, again, produce significantly better or worse results than the logistic regression models did, across different dataset sizes.

Future Work

Given these limitations, there are several directions for future research. One important extension would be to replicate this study across multiple Phishing URL datasets, such as the PhiUSIIL dataset, to evaluate the consistency of findings across different types of datasets. Additionally, comparing results across different feature representations, such as raw URLs, manually curated features, and embedding-based approaches, can provide deeper insight into how data representation interacts with dataset size to enhance detection. Further studies could also investigate the impact of dataset quality, not just quantity, by introducing controlled levels of noise or mislabeled data. Another direction of research could be analyzing the trade-off between the cost of obtaining datasets versus phishing detection accuracy, to understand how much stronger models corporations could obtain for each dollar they spend on building a dataset.

Implications

This research primarily aimed to enhance phishing detection systems, and understanding the relationship between dataset size and accuracy would help build better detection software using

larger datasets. Companies who work on building such detection software can try to obtain larger datasets to reinforce the performance of their classification algorithms as they use in live, real-world data. However, the implications of this research extend beyond phishing detection. Understanding how dataset size influences model performance is a fundamental question machine learning and can be applied to a wide range of domains, including cybersecurity, healthcare, and natural language processing. This study reinforces the idea that while more data generally leads to better performance, there are diminishing returns, and blindly increasing data size is not always the most effective strategy. Instead, a more balanced approach that considers data quality, feature representation, model hyperparameters, and computational is crucial for more effective models.

Ultimately, this research contributes to a deeper understanding of data scale impacting machine learning performance, specifically identifying the point at which additional data yields minimal benefit. By analyzing these patterns, practitioners can make more informed decisions when designing and deploying machine learning systems, leading to more efficient solutions in real-world applications.

References

- Adane, Kibreab, and Berhanu Beyene. 2022. “Machine Learning and Deep Learning Based Phishing Websites Detection: The Current Gaps and Next Directions.” *Review of Computer Engineering Research* 9 (1): 13–29. <https://doi.org/10.18488/76.v9i1.2983>.
- Aljofey, Ali, Qingshan Jiang, Abdur Rasool, Hui Chen, Wenyin Liu, Qiang Qu, and Yang Wang. 2022. “An Effective Detection Approach for Phishing Websites Using URL and HTML Features.” *Scientific Reports* 12 (1). <https://doi.org/10.1038/s41598-022-10841-5>.
- Atske, Sara, and Sara Atske. 2025. “Online Scams and Attacks in America Today.” Pew Research Center. August 14, 2025. <https://www.pewresearch.org/internet/2025/07/31/online-scams-and-attacks-in-america-today/>.
- Bitaab, Marzieh, Alireza Karimi, Zhuoer Lyu, Ahmadreza Mosallanezhad, Adam Oest, Ruoyu Wang, Tiffany Bao, Yan Shoshitaishvili, and Adam Doupé. 2025. “ScamNet: Toward Explainable Large Language Model-Based Fraudulent Shopping Website Detection.” *Proceedings of the AAAI Conference on Artificial Intelligence* 39 (27): 27841–48. <https://doi.org/10.1609/aaai.v39i27.35000>.
- Bitaab, Marzieh, Haehyun Cho, Adam Oest, Zhuoer Lyu, Wei Wang, Jorij Abraham, Ruoyu Wang, Tiffany Bao, Yan Shoshitaishvili, and Adam Doupé. 2023. “Beyond Phish: Toward Detecting Fraudulent e-Commerce Websites at Scale.” *2022 IEEE Symposium on Security and Privacy (SP)*, May, 2566–83. <https://doi.org/10.1109/sp46215.2023.10179461>.

- Devi, N. Y., C, A., P, H. S., & Coimbatore Institute of Technology. (2025). Phishing URL Detection Using Machine Learning. *International Journal for Research Trends and Innovation*, 10, a434. <https://www.ijrti.org/papers/IJRTI2509050.pdf>
- Guptta, Sumitra Das, Khandaker Tayef Shahriar, Hamed Alqahtani, Dheyaaldin Alsalman, and Iqbal H. Sarker. 2022. “Modeling Hybrid Feature-Based Phishing Websites Detection Using Machine Learning Techniques.” *Annals of Data Science* 11 (1): 217–42. <https://doi.org/10.1007/s40745-022-00379-8>.
- “How to Recognize and Avoid Phishing Scams.” 2024. Consumer Advice. October 11, 2024. <https://consumer.ftc.gov/articles/how-recognize-avoid-phishing-scams>.
- Kaitholikkal, Jishnu K S; B, Arthi (2024), “Phishing URL dataset”, Mendeley Data, V1, doi: 10.17632/vfszby9b36.1
- “New FTC Data Show a Big Jump in Reported Losses to Fraud to \$12.5 Billion in 2024.” 2025. Federal Trade Commission. July 31, 2025. <https://www.ftc.gov/news-events/news/press-releases/2025/03/new-ftc-data-show-big-jump-reported-losses-fraud-125-billion-2024>.
- Owa, K., & Adewole, O. (2025). Benchmarking machine learning techniques for phishing detection and secure URL classification. *International Journal of Computer Science and Mobile Computing*, 14(1), 20–37. <https://doi.org/10.47760/ijcsmc.2025.v14i01.003>
- Prasad, A. & Chandra, S. (2024). PhiUSIIL Phishing URL (Website) [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.1016/j.cose.2023.103545>.

Rezai, Nasrin. “Why Employees Who Self-report Are the Key to Cybersecurity | About Verizon.”

Verizon About, May 2024, www.verizon.com/about/news/dbir-2024-employer-best-practices.

Sarasjati, W., Rustad, S., Purwanto, Santoso, H. A., & Setiadi, M. (2022). Comparative Study of Classification Algorithms for Website Phishing Detection on Multiple Datasets. *2022 International Seminar on Application for Technology of Information and Communication (ISemantic)*, 448–452. <https://doi.org/10.1109/iSemantic55962.2022.9920475>

Tigerschiold, T. (2022, November 7). What is accuracy, precision, recall and F1 score? *Labelf*. <https://www.labelf.ai/blog/what-is-accuracy-precision-recall-and-f1-score>

Vajratiya Vajrobol, Gupta, B. B., & Akshat Gaurav. (2024). Mutual Information based Logistic Regression for phishing URL detection. *Cyber Security and Applications*, 100044–100044. <https://doi.org/10.1016/j.csa.2024.100044>

Appendix A – Python script for Bert Vectorization of Mendeley Phishing URL Dataset

```
import pandas as pd
import torch
import numpy as np
from transformers import DistilBertTokenizer, DistilBertModel
from tqdm import tqdm
import os
import pickle

# =====
# 1. Load Dataset
# =====
input_csv = "mendeley_phishing_dataset.csv"
df = pd.read_csv(input_csv)

urls = df["url"].astype(str).tolist()

# =====
# 2. Load DistilBERT
# =====
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

tokenizer = DistilBertTokenizer.from_pretrained("distilbert-base-uncased")
model = DistilBertModel.from_pretrained("distilbert-base-uncased")
model.to(device)
model.eval()

# =====
# 3. Chunk & Batch Settings
# =====
chunk_size = 10000 # URLs per chunk
batch_size = 32    # batch size for model
output_dir = "embeddings_chunks"
os.makedirs(output_dir, exist_ok=True)
checkpoint_file = os.path.join(output_dir, "checkpoint.pkl")

# Load checkpoint if exists
if os.path.exists(checkpoint_file):
    with open(checkpoint_file, "rb") as f:
        start_chunk = pickle.load(f)
    print(f"Resuming from chunk {start_chunk}")
```

```

else:
    start_chunk = 0

num_chunks = (len(urls) + chunk_size - 1) // chunk_size

# =====
# 4. Process chunks and save immediately
# =====
for chunk_idx in range(start_chunk, num_chunks):
    start_idx = chunk_idx * chunk_size
    end_idx = min((chunk_idx + 1) * chunk_size, len(urls))
    batch_urls_chunk = urls[start_idx:end_idx]

    chunk_embeddings = []

    # Batch-level progress bar
    for i in tqdm(range(0, len(batch_urls_chunk), batch_size), desc=f"Chunk
{chunk_idx} ({start_idx}-{end_idx})"):
        batch_urls = batch_urls_chunk[i:i+batch_size]

        encoded = tokenizer(
            batch_urls,
            padding=True,
            truncation=True,
            return_tensors="pt",
            max_length=64
        )
        encoded = {k: v.to(device) for k, v in encoded.items()}

        with torch.no_grad():
            outputs = model(**encoded)

        cls_embeddings = outputs.last_hidden_state[:, 0, :]
        chunk_embeddings.append(cls_embeddings.cpu().numpy())

    # Save chunk immediately
    chunk_file = os.path.join(output_dir, f"chunk_{chunk_idx}.npy")
    np.save(chunk_file, np.vstack(chunk_embeddings))

    # Update checkpoint
    with open(checkpoint_file, "wb") as f:
        pickle.dump(chunk_idx + 1, f)

```

```

    print(f"Saved chunk {chunk_idx} embeddings")

# =====
# 5. Combine all chunks into one final file
# =====
all_embeddings = []

for chunk_idx in range(num_chunks):
    chunk_file = os.path.join(output_dir, f"chunk_{chunk_idx}.npz")
    all_embeddings.append(np.load(chunk_file))

full_embeddings = np.vstack(all_embeddings)
np.save("distilbert_embeddings_full.npy", full_embeddings)

print("All chunks combined into distilbert_embeddings_full.npy")

# Remove checkpoint if done
if os.path.exists(checkpoint_file):
    os.remove(checkpoint_file)

```

Appendix B – Python script for training and testing ML models (Example)

```

import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# =====
# Constants
# =====

NUM_ROWS = 300000
TRAIN_TEST_RANDOM_STATE = 41

# =====

```

```

# Load and preprocess embeddings and labels
# =====
embeddings_file = "distilbert_embeddings_full.npy"
X = np.load(embeddings_file)

df = pd.read_csv("mendeley_phishing_dataset.csv")
label_map = {"phishing": 1, "legitimate": 0}
y = df["type"].map(label_map).values

# =====
# APPENDIX B.i. Select Subset of data
# =====
indices = np.random.choice(len(X), size=NUM_ROWS, replace=False)
X = X[indices]
y = y[indices]

# =====
# APPENDIX B.ii. Train/test split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=TRAIN_TEST_RANDOM_STATE, stratify=y
)
print(f"Training on {X_train.shape[0]} samples, testing on {X_test.shape[0]} samples")

# =====
# APPENDIX B.iii. Train Logistic Regression
# =====
clf = LogisticRegression(
    solver="lbfgs",
    max_iter=1000,
)
clf.fit(X_train, y_train)

# =====
# APPENDIX B.iv. Evaluate
# =====
y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"\nTest Accuracy: {accuracy:.4f}")

print("\nClassification Report:")

```

```
print(classification_report(y_test, y_pred, digits=4))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Appendix C – Machine Learning Models’ metrics

Dataset Size	Random State	Accuracy	Precision	Recall	F1-score
1000	13	0.98	0.979951	0.98	0.979831
1000	24	0.985	0.984947	0.985	0.984938
1000	41	0.975	0.974863	0.975	0.974896
1000	45	0.98	0.979952	0.98	0.979831
1000	91	0.97	0.97	0.97	0.97
5000	23	0.99	0.989983	0.99	0.989969
5000	27	0.985	0.985083	0.985	0.98487
5000	38	0.993	0.993063	0.993	0.992962
5000	79	0.99	0.989983	0.99	0.989969
5000	96	0.989	0.988975	0.989	0.988975
10000	13	0.993	0.993063	0.993	0.992962
10000	14	0.9925	0.992505	0.9925	0.992474
10000	21	0.9915	0.991489	0.9915	0.991477
10000	64	0.9895	0.989514	0.9895	0.989447
10000	85	0.992	0.991996	0.992	0.991976
25000	21	0.9924	0.992395	0.9924	0.992379
25000	37	0.993	0.99301	0.993	0.992976
25000	39	0.994	0.994008	0.994	0.993982
25000	74	0.9924	0.992392	0.9924	0.992382
25000	87	0.993	0.993005	0.993	0.992978
50000	13	0.9934	0.99342	0.9934	0.993375
50000	35	0.9934	0.993405	0.9934	0.99338
50000	81	0.9944	0.9944	0.9944	0.994387
50000	93	0.9933	0.993303	0.9933	0.99328
50000	99	0.9945	0.994503	0.9945	0.994486
100000	38	0.9955	0.995503	0.9955	0.995491
100000	63	0.99515	0.99515	0.99515	0.99514
100000	67	0.99385	0.993846	0.99385	0.993837
100000	79	0.9953	0.995296	0.9953	0.995295
100000	85	0.9943	0.994301	0.9943	0.994286
200000	10	0.995725	0.995722	0.995725	0.995719
200000	30	0.994975	0.994974	0.994975	0.994965
200000	45	0.995325	0.995326	0.995325	0.995316
200000	64	0.995975	0.995978	0.995975	0.995967

200000	99	0.99525	0.995252	0.99525	0.99524
300000	29	0.994283	0.994278	0.994283	0.994274
300000	37	0.995533	0.995534	0.995533	0.995525
300000	45	0.995617	0.995616	0.995617	0.995609
300000	53	0.99495	0.994947	0.99495	0.994942
300000	91	0.995283	0.995281	0.995283	0.995275
450176	21	0.995624	0.995621	0.995624	0.995618
450176	22	0.995113	0.995112	0.995113	0.995104
450176	23	0.995124	0.995121	0.995124	0.995117
450176	55	0.996057	0.996056	0.996057	0.996051
450176	58	0.995668	0.995667	0.995668	0.995662

Appendix D – Post-hoc test p-values

Table 2	
Dataset Sizes	F1 p-value
5000 - 1000	<0.01
10000 - 1000	<0.01
25000 - 1000	<0.01
50000 - 1000	<0.01
100000 - 1000	<0.01
200000 - 1000	<0.01
300000 - 1000	<0.01
450176 - 1000	<0.01
10000 - 5000	0.77
25000 - 5000	0.24
50000 - 5000	0.07
100000 - 5000	0.01
200000 - 5000	<0.01
300000 - 5000	0.01
450176 - 5000	<0.01
25000 - 10000	0.99
50000 - 10000	0.85
100000 - 10000	0.4
200000 - 10000	0.19
300000 - 10000	0.28
450176 - 10000	0.17
50000 - 25000	1.00

100000 - 25000	0.91
200000 - 25000	0.69
300000 - 25000	0.82
450176 - 25000	0.66
100000 - 50000	1.00
200000 - 50000	0.96
300000 - 50000	0.99
450176 - 50000	0.94
200000 - 100000	1.00
300000 - 100000	1.00
450176 - 100000	1.00
300000 - 200000	1.00
450176 - 200000	1.00
450176 - 300000	1.00